

EXPERIMENT NO. 4:

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim:

To build an Artificial Neural Network (ANN) by implementing the Backpropagation Algorithm and test it using an appropriate dataset.

Procedure:

1. Import the required Python libraries.
2. Define the input features and target outputs.
3. Initialize the weights randomly.
4. Apply the sigmoid activation function.
5. Perform forward propagation to compute the network output.
6. Calculate the error between the predicted and actual outputs.
7. Apply the backpropagation algorithm to update the weights.
8. Repeat the training process for a fixed number of epochs.
9. Display the final predicted output after training.

Program:

Artificial Neural Network using Backpropagation

import numpy as np

Input dataset (XOR Problem)

X = np.array([[0,0],

[0,1],

[1,0],

[1,1]])

Output dataset

y = np.array([[0],

[1],

[1],

[0]])

Sigmoid Activation Function

```

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Seed for reproducibility
np.random.seed(1)

# Initialize weights
input_neurons = 2
hidden_neurons = 4
output_neurons = 1

weights_input_hidden = np.random.uniform(size=(input_neurons, hidden_neurons))
weights_hidden_output = np.random.uniform(size=(hidden_neurons, output_neurons))

# Training parameters
epochs = 10000
learning_rate = 0.5

# Training using Backpropagation
for epoch in range(epochs):
    # Forward Propagation
    hidden_input = np.dot(X, weights_input_hidden)
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, weights_hidden_output)
    predicted_output = sigmoid(final_input)

    # Error Calculation
    error = y - predicted_output

    # Backpropagation
    d_predicted_output = error * sigmoid_derivative(predicted_output)
    error_hidden = d_predicted_output.dot(weights_hidden_output.T)
    d_hidden_layer = error_hidden * sigmoid_derivative(hidden_output)

    # Update Weights
    weights_hidden_output += hidden_output.T.dot(d_predicted_output) * learning_rate

```

```
weights_input_hidden += X.T.dot(d_hidden_layer) * learning_rate

# Display Results

print("Expected Output:")

print(y)

print("\nPredicted Output:")

print(np.round(predicted_output, 3))
```

Output:

```
... Expected Output:
[[0]
 [1]
 [1]
 [0]]

Predicted Output:
[[0.03 ]
 [0.972]
 [0.974]
 [0.025]]
```

Result:

The Artificial Neural Network (ANN) was successfully implemented using the Backpropagation Algorithm. The network was trained using the XOR dataset and accurately predicted the output values after training. The predicted outputs closely matched the expected outputs, demonstrating the successful learning capability of the backpropagation algorithm.